

郭熙

Agent 后端应用开发

14727656774 | 485222758@qq.com

求职方向: Agent 应用 / LLM 工程 | github.com/Gurp-S



教育背景

2023.09-2027.06 上海海洋大学 (双一流)

计算机类 / 空间信息与数字技术 (本科)

项目经历

XEYO — 自研桌面 Agent 引擎

独立完成 2026.05-2026.07

用 Python 从零实现的桌面 AI Agent 引擎 (339 个测试文件), 涵盖主循环、工具系统、上下文工程、记忆与权限安全, 并配有独立桌面端与终端界面。

实测 编码 Agent 公开基准 Terminal-Bench 2.1, deepseek-v4-flash 同模型: 官方中立 scaffold 全量 157 次 trial 都打不下的 32 道硬题, 自研引擎攻下 13 道 (40.6%, 官方 0%)

- **动态上下文注入管线**: Agent 每轮要注入的状态差异极大 —— 用户指令 / 引擎当前态 / 引擎事件, 三类语义分开: **用户输入进历史**、**状态幂等重注** (值不变不发)、**事件 drain** (永不裁剪、门控、去重 —— **去重即静默丢事件**)。全部注入作为一条原生 system 消息追加于投影末尾, KV 前缀逐字节不变 —— **5.31 亿 token 真实账本: 长上下文缓存命中中位 98.54%、P90 99.58%**。
- **自研压缩方法 WSC**: 把执行历史当**工作流图**压缩 —— 按**依赖可达性**决定保留什么 (对比引擎既有的「位置 + KV 经济」口径), 剪枝不删除而是降级为「缺口 + 可展开句柄」, **零 LLM、两跑逐字节一致**。在 **317 个真实会话 (119 MB)** 离线回放: 压缩率中位 **82.3%**、长会话 **87.6%**, 热层 **24,983 → 1,571 token**; **22,472 个被剪节点句柄覆盖 100%**、往返无损 **35,413/35,413**; LCP 探针定位到命中率崩塌根因 —— 据此判定暂不并入主链。
- **断点续跑与可逆回滚**: 解决"Agent 跑到第 30 轮崩了、改错文件撤不回来" —— 文件快照走**内容寻址** (日志只存哈希、不存正文); 回滚不是重放而是**条件写**: 先校验当前文件哈希与预期一致, 不符即**抛冲突、绝不 force**, 并要求同一计划哈希 + 未过期 + 同一版本号才放行, 执行前再拍检查点保证**幂等重试**。
- **工程配套**: 把 **45 个模型可见文本面**逐字节钉成 golden 基线 (**1 字符**改动即被检出, 并自带"注入变异断言 100% 检出"的元测试); 输出**按形态压缩而非截断** (pytest **1126 → 78 token**), AST 符号索引让读一个函数从 **20,523 → 206 token**、符号召回 **100% vs 正则 51%**。
- **子代理运行时**: 多智能体最容易在三个地方翻车 —— 停止时子代理还在跑 (**资源泄漏**)、工人的中间过程污染主会话 (**越跑越贵**)、并发结算竞态 (**状态错乱**)。解法是一句可复述的判断: **共享取消、不共享历史** —— 取消是全局意志, 历史是局部记录。据此: 子代理跑独立循环与独立存储, 中间过程写侧链、不回主上下文; 但**复用父级取消**, 停止能取消全部; 主 → 子消息**只暂存不打断** (往正在执行的计划里插消息会破坏它); 结算汇聚到**单一槽位**后按三个租户分发, **租户异常互相隔离**; 工具面由引擎**裁剪后下发**。

RXY-Agent — Java 企业级 RAG 系统

独立完成 2026.04-2026.05

用 Java 从零实现的企业级 RAG 系统, 覆盖 ETL 管道、多通道召回、评估闭环与万级并发, 每个环节多层降级、全链路可追踪。

实测 CRAG (Meta KDD 2024) 60 题 5 域: Evidence Hit@8 **0.967 (58/60)** / MRR **0.958**, 多跳 comparison 类 **1.000** || 并发压测 (单机): **10,000 并发连接 0 服务端错误**、聚合 **~4,951 RPS** (p50 1.3s); 峰值 **11,200 连接 / ~6,028 RPS**

- **检索质量工程**: 解决同主题双文档的文件级混淆与召回不准 —— 重排输入改为「本文档《文档名》章节「章节路径」+ 原文」 (两字段均为**入库期元数据, 运行时零成本**); 按文档分组做置信度门控, **非最优文档候选项 $\geq$ 最优文档分 $\times 0.9$** (阈值经 0.8/0.9/0.95 三点扫描取优), 否则整组剔除、最优文档全留; 最终分按 **$0.2 \times \text{向量} + 0.6 \times \text{重排} + 0.2 \times \text{BM25}$** —— 黄金集 20 例平均精度 **0.769 → 0.954** (含一次 embedding 升级), 混淆残余降至 **2 例**。
- **确定性证据门控与记忆分流**: 解决"库外问题硬答幻觉"与"拒答一刀切误伤" —— 证据充分性为**纯数值判定**、**全程不调 LLM**: **证据分 = 融合最高分 $\times (0.7 + 0.3 \times \min(1, \text{支撑块数} / 3))$** , 阈值 **0.45**; 不足则**触发一次免 LLM 重检索**, 仍不足再按长期记忆召回分流。长期记忆同属**零 LLM 依赖**: 11 条正则抽取 + 重要度打分, **bigram Jaccard ≥ 0.8** 去重, 超 300 条按 **重要度 $\times (1 + \log_2 \text{命中次数})$** 淘汰; 召回按 **$0.45 \times \text{相关度} + 0.35 \times \text{重要度} + 0.20 \times \text{时间衰减}$** (阈值 **0.22**)。
- **并发与性能**: 解决高并发下的尾延迟尖刺 —— Tomcat 与业务 IO 统一到虚拟线程; Kafka 拆双 producer (ETL 走 **acks=all + 幂等 + 重试 10 次**, trace 走 **acks=1 + linger 20ms + batch 128KB**); 4000 并发首测暴露**服务端 7900 次 Neo4j 连接池排队满**, 定位到实体识别**每请求打两次图库**, 改为启动时全量载入实体缓存、池容量 100→300、获取超时 60s→5s —— **排队归零**; 外部 rerank 加 **5 万条 / 10 分钟结果缓存**与三级降级 —— **200 并发服务端检索步 p95 由 2,222ms 降至 404ms**。

专业技能

- **AI Agent 系统开发**: 深入理解 Agentic 循环与状态机设计, 具备从 0 到 1 构建高稳定 Agent 引擎的经验; 熟悉 LangChain、LangGraph, 掌握 ReAct 推理、工具调用与任务规划; 对多智能体协作、MCP 协议及 Human-in-the-Loop 有工程实践。
- **RAG 与知识工程**: 掌握 RAG 全流程设计与优化: 文档 ETL (Tika / 语义分块)、向量数据库 (Milvus)、混合检索 (向量 + BM25 + 知识图谱)、重排序 (Cross-Encoder) 与 Query 改写; 具备多级记忆管理与上下文工程实战经验。
- **大模型工程化**: 熟练对接 DeepSeek、OpenAI 等 LLM API 与 Function Calling、提示词工程、流式输出; 掌握多模型路由、熔断降级、异步预取, 擅长调用成本优化与输出确定性提升。
- **后端开发与工程化**: 以 Java 为主力语言、Python 用于 AI 原型开发; 用 Spring Boot 3 构建高可用服务, 掌握 MySQL (索引优化 / 事务)、Redis (缓存策略 / ZSet)、Kafka (异步解耦); 具备熔断降级、链路追踪与 Docker 容器化部署能力。
- **安全与可靠性**: 具备 Deny-first 权限裁决与提示注入防御经验; 注重系统的确定性与可观测性, 强于失败案例归因与系统韧性设计。